CineDesk — Login e Cadastro

Guia de estudo: como executar, a lógica das telas, o DAO, hash de senha e banco de dados

1. Visão geral do projeto

O CineDesk é um aplicativo em Java Swing com duas telas, login e cadastro, ligadas por uma classe DAO que guarda os usuários. Os arquivos ficam na pasta **CineDesk/**:

Arquivo	Função
TelaLogin.java	Tela de login: lê usuário e senha e pede ao DAO para conferir.
TelaCadastro.java	Tela de cadastro: valida os campos e pede ao DAO para salvar.
UsuarioDAO.java	Acesso aos dados: salva, busca e autentica usuários (arquivo usuarios.dat).
Usuario.java	Modelo: guarda nome, login, salt e hash da senha.

2. Como executar

Requisito: JDK 8 ou mais novo. O código foi compilado e testado com o **Java 21**. Para ver a versão instalada:

```
java -version
```

Na pasta do projeto, compile e rode:

```
cd TelasCine/CineDesk
javac *.java
java TelaLogin
```

Na IDE (IntelliJ, Eclipse, NetBeans): abra a pasta, clique com o botão direito em **TelaLogin.java** e escolha **Run**.

Testando o fluxo

- Não existe usuário pronto: primeiro crie um na tela de **Cadastro**.
- Exemplo: Nome Maria Silva, Usuário maria, Senha 1234, Confirmar 1234.
- Clique em **Cadastrar**, depois em **Login**, digite maria / 1234.
- Deve aparecer: "Login realizado com sucesso! Bem-vindo, Maria Silva."
- A conta fica salva em **usuarios.dat**; para apagar todos os usuários, apague esse arquivo.

3. Onde está a lógica

A lógica está dividida em três partes, cada uma com sua responsabilidade:

Classe	Tipo de lógica	O que faz
TelaCadastro / TelaLogin	Lógica da tela	Lê o que foi digitado, valida campos vazios, senhas diferentes, senha curta, e mostra as mensagens.
UsuarioDAO	Lógica de dados	Salva no arquivo, verifica se o usuário já existe, gera o hash da senha e confere usuário e senha.
Usuario	Modelo	Só carrega os dados: nome, login, salt e hash da senha.

Caminho do login

- A **TelaLogin** pega usuário e senha e confere se não estão vazios.
- Chama `usuarioDAO.autenticar(usuario, senha)`.
- O **UsuarioDAO** lê o usuarios.dat, procura o usuário e compara a senha.
- Devolve um `Usuario` se estiver certo, ou `null` se estiver errado.
- A tela mostra "Login realizado com sucesso!" ou "Usuário ou senha inválidos".

Validações do cadastro

- Todos os campos preenchidos.
- Senha e confirmação iguais.
- Usuário sem espaços.
- Senha com pelo menos 4 caracteres.
- Usuário ainda não cadastrado (não diferencia maiúsculas de minúsculas).

Por que separar assim (padrão DAO)

A tela não sabe onde os dados estão guardados, e o DAO não sabe nada de janelas. Se trocar o arquivo por um banco de dados, **só o UsuarioDAO muda**; as telas continuam iguais.

4. O que é o hash da senha

A coluna **hash_senha** guarda a senha embaralhada, nunca a senha de verdade. Um hash é uma função matemática (aqui, **SHA-256**) que transforma um texto numa sequência de tamanho fixo:

```
"1234"  ->  SHA-256  ->  "A6xnQhbz4Vx2HuGl4lXwZ5U2I8iziLRFnhP5eNfIRvQ="
```

- **Mão única:** a partir do hash não dá para descobrir a senha.
- **Sempre o mesmo resultado:** a mesma entrada gera sempre o mesmo hash.
- **Qualquer mudança altera tudo:** "1235" gera um hash completamente diferente.

Se alguém invadir o banco ou abrir o arquivo, vê só o hash, e não a senha. Isso protege o usuário também em outros sites onde ele use a mesma senha.

O salt

É um valor aleatório criado para cada usuário e misturado com a senha antes de gerar o hash: `hash = SHA-256(salt + senha)`. Assim, duas pessoas com a mesma senha têm hashes diferentes, e tabelas prontas de hashes de senhas comuns não funcionam. Exemplo salvo para a usuária ana, senha 1234:

```
salt:       MhW84EfhPhiTha3Bywotfw==
hash_senha: 8jXrIyQfKAkJxJgc0sWldvnXUVUXyuWE1L4x+TFwHAw=
```

Como o login confere a senha

- A pessoa digita a senha.
- O DAO pega o salt salvo daquele usuário.
- Calcula SHA-256(salt + senha digitada).
- Se o resultado for igual ao hash_senha salvo, a senha está certa.

No código: `gerarHash(...)` calcula o hash e `autenticar(...)` faz a comparação, ambos em UsuarioDAO.java. Em sistemas profissionais se usam algoritmos mais lentos de propósito, como BCrypt ou PBKDF2; para um trabalho escolar, SHA-256 com salt já mostra bem o

conceito.

5. Por que a classe Usuario só tem get

A classe é **imutável**: os valores entram uma única vez, pelo construtor, e depois não mudam.

```
private final String login;   // "final" = não pode ser alterado depois

public Usuario(String nome, String login, String salt, String hashSenha) {
    this.login = login;       // o valor é definido aqui, uma vez só
    ...
}

public String getLogin() {    // só leitura
    return login;
}
```

- **O login não deve mudar:** ele identifica o usuário.
- **Segurança:** nenhuma parte do código troca sem querer o hash de um usuário já autenticado.
- **Nada no projeto precisa de set hoje:** o DAO já cria o objeto completo.

Quando usar set

Se houver uma tela de **editar perfil**, ou se o professor pedir o padrão **JavaBean** (construtor vazio, get e set para cada campo), comum em aulas de DAO com JDBC:

```
Usuario u = new Usuario();
u.setNome(rs.getString("nome"));
u.setLogin(rs.getString("login"));
```

Os dois jeitos estão corretos: o imutável é mais seguro, e o JavaBean é mais flexível.

6. Ligando no banco de dados

Hoje o UsuarioDAO salva no arquivo **usuarios.dat**, não num banco. A estrutura já está pronta para a troca, porque as telas só chamam `cadastrar(...)` e `autenticar(...)`. Passos (exemplo com MySQL):

1. Criar a tabela

```
CREATE TABLE usuarios (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    nome       VARCHAR(100) NOT NULL,
    login      VARCHAR(50)  NOT NULL UNIQUE,
    salt       VARCHAR(64)  NOT NULL,
    hash_senha VARCHAR(64)  NOT NULL
);
```

2. Criar uma classe Conexao

Ela abre a conexão com `DriverManager.getConnection(url, usuario, senha)`.

3. Reescrever o UsuarioDAO com SQL

- cadastrar: `INSERT INTO usuarios (nome, login, salt, hash_senha) VALUES (?, ?, ?, ?)`

- buscarPorLogin: `SELECT * FROM usuarios WHERE login = ?`
- A parte do hash da senha continua igual.

4. Adicionar o driver

Para MySQL: **mysql-connector-j-x.x.x.jar** nas bibliotecas da IDE ou pelo Maven.

Cuidados

- Use sempre **PreparedStatement** com ?, nunca SQL juntando textos (evita SQL Injection).
- Feche as conexões com **try-with-resources**.
- Os outros DAOs do projeto (Filmes, Sessões etc.) devem usar a mesma classe Conexao.
- Os nomes da tabela e das colunas precisam bater com o banco definido pelo grupo ou professor.

7. O que ainda falta ou pode melhorar

#	Item	Importância
1	Depois do login não abre nenhuma tela: aparece só a mensagem e a janela de login continua aberta. Falta uma tela principal.	Alta
2	Na tela de cadastro, a tecla Enter não aciona o botão Cadastrar.	Baixa
3	O usuarios.dat é criado na pasta de onde o programa é rodado; rodando de outra pasta, os usuários parecem sumir.	Média
4	Quando o cadastro é recusado por validação, a senha não é apagada da memória.	Baixa
5	Não há limite de tentativas de login erradas.	Baixa
6	Não há opção de sair da conta nem de recuperar a senha.	Depende do trabalho